

МІНІСТЕРСТВО ОСВІТИ І НАУКИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА
ФРАНКА
Факультет прикладної математики та інформатики
Кафедра програмування

Лабораторна робота №9
Графічні операції

Виконала:
Студентка групи ПМІ-46
Тригуб Марта

Мета роботи

Розширити графічний редактор, розроблений у лабораторній роботі №8, шляхом реалізації трьох додаткових графічних операцій: скасування малювання клавішею Esc під час виконання операції; повноцінного Undo/Redo на основі стеку знімків стану; інструменту виокремлення фігур із підтримкою переміщення, копіювання та вставки.

Теоретичні відомості

Механізм Undo/Redo для векторної графіки

Оскільки в роботі використовується векторна графіка – зображення зберігається у вигляді списку фігур із параметрами – реалізація Undo/Redo є досить природньою. Перед кожною деструктивною операцією (малювання, переміщення, видалення, вставка) поточний список фігур копіюється і зберігається у стек Undo. Операція Undo відновлює попередній знімок, зберігаючи поточний у стек Redo. Стек Undo обмежений 50 рівнями, щоб обмежити споживання пам'яті.

Порівняно з растровою графікою, де перед кожною операцією доводиться копіювати цілий bitmap (масив пікселів), векторний підхід є значно ефективнішим за пам'яттю: кожен знімок містить лише список координат і параметрів фігур.

Скасування незавершеного малювання (Esc)

Під час малювання фігури (між подіями MouseDown і MouseUp) на canvas присутній тимчасовий об'єкт-preview, помічений тегом 'preview'. Натискання Esc видаляє цей об'єкт і скидає прапорець drawing = False, не додаючи фігуру до списку shapes. Таким чином фігура не зафіксується і не потрапить у стек Undo.

Інструмент виокремлення і переміщення

Інструмент 'select' дозволяє виокремлювати фігури двома способами: одиночним кліком по фігурі або виокремленням прямокутною рамкою. Пунктирна синя рамка, побудована методом create_rectangle із параметром dash, візуально позначає виокремлену область. Переміщення виокремлених фігур відбувається через canvas.move() при кожній події B1-Motion, що забезпечує плавний drag-and-drop.

Опис програмної реалізації

Нові поля стану класу

У порівнянні з лаб. №8 клас GraphicsEditor доповнено такими полями:

Поле	Тип	Призначення
_undo_stack	list[list]	Стек знімків для Undo (до 50 рівнів)
_redo_stack	list[list]	Стек знімків для Redo
drawing	bool	True, якщо зараз виконується малювання
moving	bool	True, якщо зараз виконується переміщення
selected_idx	list[int]	Індекси виокремлених фігур у self.shapes
sel_rect_id	int None	canvas-id пунктирної рамки виокремлення
move_last_x/y	int	Попередня позиція миші при переміщенні

Скасування малювання клавішею Esc

Обробник on_escape() перевіряє прапорець drawing. Якщо малювання в процесі – видаляє preview-об'єкт та скидає стан:

```
def on_escape(self):  
    if self.drawing:  
        self.drawing = False  
        if self.preview_id:  
            self.canvas.delete(self.preview_id)  
            self.preview_id = None  
        if self.sel_rect_id:  
            self.canvas.delete(self.sel_rect_id)  
            self.sel_rect_id = None  
        self.status_var.set("Малювання скасовано (Esc)")  
    elif self.moving:  
        self.moving = False  
    elif self.selected_idx:  
        self.edit_deselect()
```

Метод прив'язаний до події <Escape> через root.bind.

Undo/Redo

Метод `_push_undo()` викликається перед кожною деструктивною операцією. Він серіалізує поточний список фігур (без поля `canvas_id`) за допомогою `copy.deepcopy` і кладе знімок у стек:

```
def _push_undo(self):
    """Зберегти поточний стан у стек Undo."""
    snapshot = copy.deepcopy([
        {k: v for k, v in s.items() if k != "canvas_id"}
        for s in self.shapes
    ])
    self._undo_stack.append(snapshot)
    if len(self._undo_stack) > 50:
        self._undo_stack.pop(0)
```

Метод `undo()` переміщує поточний стан у стек Redo, відновлює попередній знімок і перемальовує canvas методом `_redraw_all()`. Метод `redo()` виконує зворотню операцію.

Виокремлення, копіювання, переміщення

При активному інструменті 'select' логіка `on_mouse_down` розгалужується: якщо клік на фігуру – починається режим переміщення (`moving = True`), інакше – малювання рамки виокремлення:

```

def on_mouse_down(self, event):
    x, y = event.x, event.y

    if self.current_tool == "select":
        # чи натиснули на існуючу фігуру?
        hit = self._hit_shape(x, y)
        if hit is not None:
            if hit not in self.selected_idx:
                self.selected_idx = [hit]
                self._draw_selection_rect()
            # починаємо переміщення
            self.moving = True
            self.move_last_x, self.move_last_y = x, y
        else:
            # починаємо виокремлення прямокутником
            self.edit_deselect()
            self.drawing = True
            self.start_x, self.start_y = x, y
    else:
        # звичайне малювання
        self.drawing = True
        self.start_x, self.start_y = x, y
        self.preview_id = None

```

При `on_mouse_move` в режимі `moving` викликається `_move_selected(dx, dy)`, який зсуває координати у `self.shapes` і переміщує `canvas`-об'єкти через `canvas.move()`. Після відпускання кнопки викликається `_push_undo()`.

Метод `_draw_selection_rect()` обчислює `bounding box` усіх виокремлених фігур і малює навколо нього пунктирну рамку синього кольору:

```
def _draw_selection_rect(self):
    """Намалювати пунктирну рамку навколо всіх виокремлених фігур."""
    if self.sel_rect_id:
        self.canvas.delete(self.sel_rect_id)
        self.sel_rect_id = None
    if not self.selected_idx:
        return
    xs, ys = [], []
    for i in self.selected_idx:
        s = self.shapes[i]
        xs += [s["x1"], s["x2"]]; ys += [s["y1"], s["y2"]]
    pad = 4
    self.sel_rect_id = self.canvas.create_rectangle(
        min(xs)-pad, min(ys)-pad, max(xs)+pad, max(ys)+pad,
        outline=SEL_COLOR, dash=SEL_DASH, width=2, tags=("selection",))
    self.canvas.tag_raise("selection")
```

Перелік гарячих клавіш

Клавіша	Дія
Ctrl+Z	Undo — скасувати останню операцію
Ctrl+Y	Redo — повернути скасовану операцію
Esc	Скасувати малювання / зняти виокремлення
Ctrl+A	Виокремити всі фігури
Ctrl+C / Ctrl+X	Копіювати / вирізати виокремлені фігури
Ctrl+V	Вставити зі зсувом +20px
Delete	Видалити виокремлені фігури
Ctrl+S	Зберегти файл

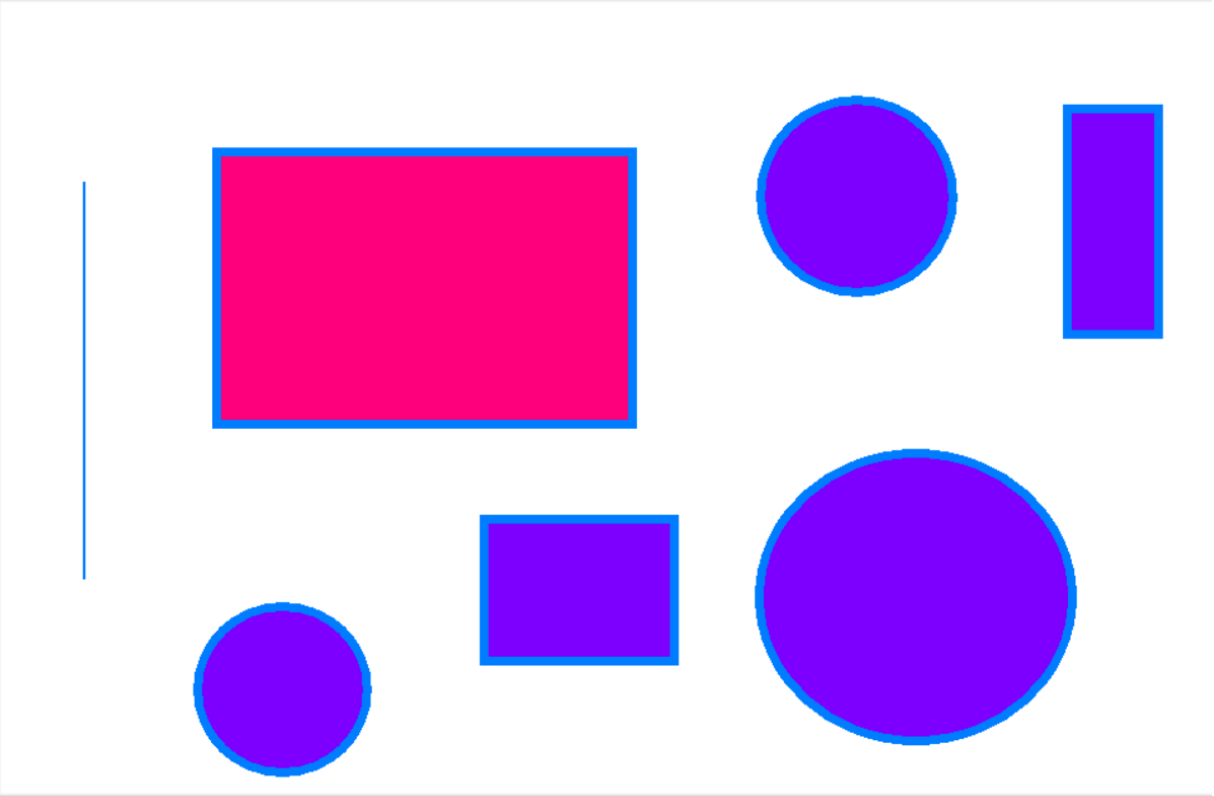
Тестування

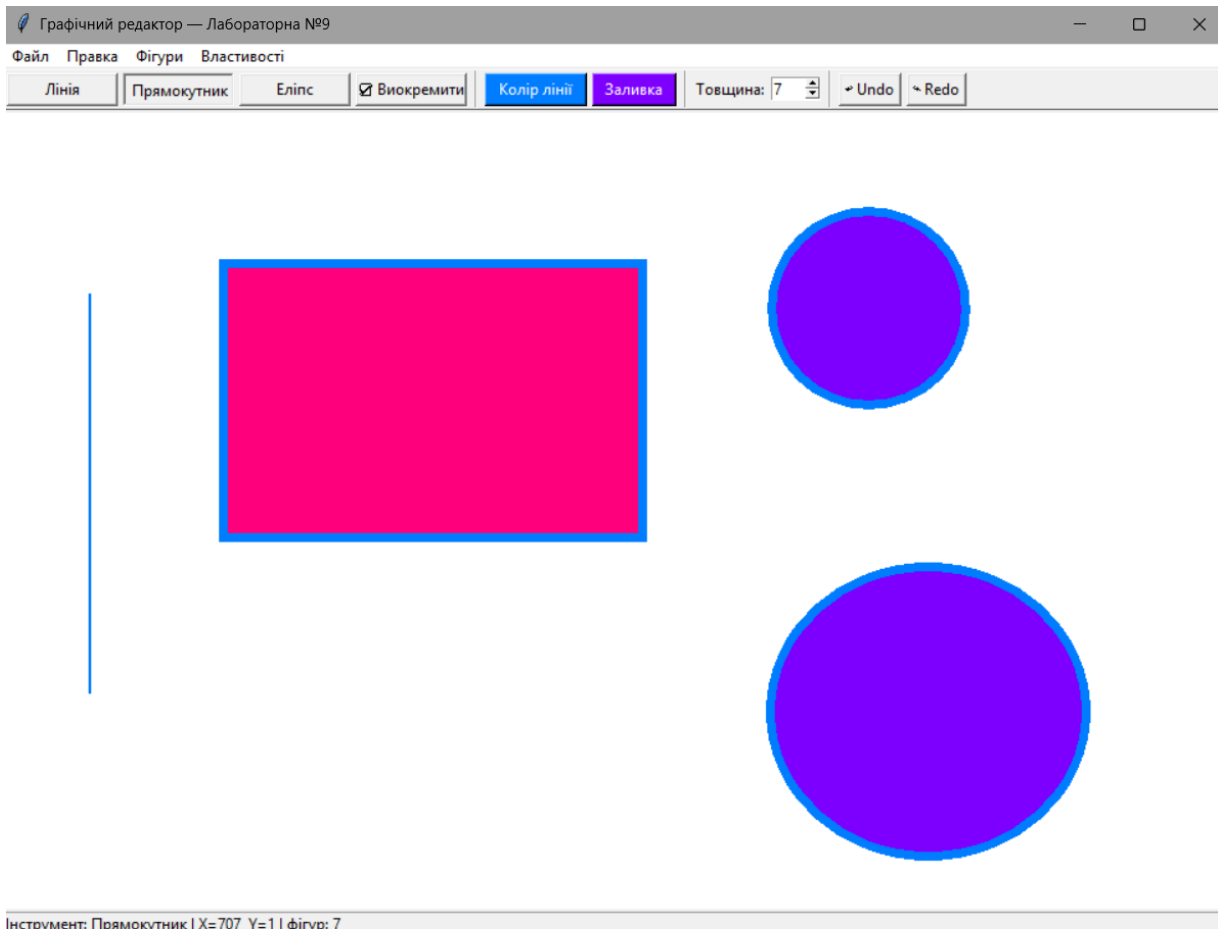
Для перевірки всіх нових операцій виконано такі тестові сценарії:

1. Намалювати прямокутник, в процесі малювання натиснути Esc – фігура не зафіксувалась, полотно залишилось без змін.
2. Намалювати 3 фігури -> Ctrl+Z тричі -> полотно повернулося до початкового стану. Ctrl+Y тричі – фігури повернулися.
3. Активувати інструмент 'Виокремити', клікнути на еліпс – з'явилася синя пунктирна рамка. Перетягнути мишею – еліпс

переміщується плавно. Після відпускання Undo відновлює початкову позицію.

4. Виокремити групу фігур рамкою -> Ctrl+C -> Ctrl+V -> скопійовані фігури з'явилися зі зсувом +20px і одразу виокремлені.
5. Виокремити дві фігури -> Delete -> фігури видалені -> Ctrl+Z -> фігури відновлені.
6. Зберегти малюнок у JSON -> відкрити -> всі фігури відтворені коректно.





Всі сценарії виконано успішно. Undo/Redo коректно відстежує стан при малюванні, переміщенні, видаленні та вставці. Esc надійно скасовує незавершене малювання, не порушуючи вже намальовані фігури.

Висновки

У ході лабораторної роботи графічний редактор розширено трьома групами операцій. По-перше, реалізовано скасування поточного малювання натисканням Esc шляхом видалення тимчасового об'єкту-preview і скидання прапорця drawing. По-друге, додано повноцінний механізм Undo/Redo на основі стеку знімків списку фігур (copy.deerscopy), який охоплює всі деструктивні операції. По-третє, реалізовано інструмент виокремлення фігур з підтримкою drag-and-drop переміщення, групового копіювання та вставки.

Векторний підхід до збереження зображень довів свою перевагу порівняно з растровим: знімки стану займають мінімум пам'яті, а перемальовування canvas є швидким і безартефактним. Отримані навички є основою для реалізації більш складних графічних редакторів у подальших лабораторних роботах.